

Step 1 — Map existing conventions. Before writing anything, read the project's own contribution files:

- CONTRIBUTING.md — coding style, PR format, commit conventions, review process
- AGENTS.md / CLAUDE.md — AI usage policy and agent instructions (if present)
- Any linter/formatter configs (.eslintrc, rubocop.yml, gofmt settings, etc.)
- The existing "Definition of Done": the activities required to solve an issue (look in CONTRIBUTING.md, official documentation, PR templates, or issue tracking/triage guidelines)

Summarize what the project already requires. This becomes the baseline your standards document adopts.

Summary of AGENTS.md

Outline is a collaborative knowledge base for teams, built with React and TypeScript in both frontend and backend. It uses a real time collaboration engine, and is designed for excellent performance and user experience. The backend is a Koa server with an RPC API and uses PostgreSQL and Redis. The application can be self-hosted or used as a cloud service. It needs these dependencies installed before running

- Use yarn for all dependency management.
- After updating dependency versions, install to update lockfiles

Summary of ARCHITECTURE.md

Outline has a backend and frontend codebase in this monorepo, both are written in TypeScript and share some code. Outline's frontend is a React application compiled with Vite. It uses MobX for state management and Styled Components for component styles. Unless global, state logic and styles are always co-located with React components together with their subcomponents to make the component tree easier to manage. The API server is driven by Koa, it uses Sequelize as the ORM and Redis with Bull for queues and async event management. Authorization logic is contained in cancan policies under the "policies" directory.

Step 2 — Adopt, adapt, supplement. For each area below, either adopt the project's existing standard, note any team-specific adaptations, or fill the gap where the project is silent:

Section	What to cover
Coding conventions	Guidelines we kept from Agents.md - Use early returns for readability. - Emphasize type safety and static analysis.

-
- Do not replace smart quotes (""") or (") with simple quotes (").
 - Do not add translation strings manually; they will be extracted automatically from the codebase.

TypeScript Usage

- Avoid "unknown" unless absolutely necessary.
- Never use "any".
- Prefer type definitions; avoid type assertions (as, !).
- Always use curly braces for if statements.
- Avoid # for private properties.
- Prefer interface over type for object shapes.

Code Quality

- Use Oxlint for linting: yarn lint
- Format code with oxfmt: yarn format
- Check types with TypeScript: yarn tsc

Error Handling

- Use custom error classes in server/errors.ts.
- Always catch and handle errors appropriately.
- Log errors with appropriate context.
- Return user-friendly error messages.
- Never expose sensitive information in errors.

Branching and commit conventions

Branch naming: issue-<issue_name> or feature-<feature_name>
Commit message format: one sentence description of what added
in past tense:

Pull request process

Before submitting a pull request you must discuss with the core team by creating or commenting in an issue on GitHub. Then at least one person will review the code, and if there are any suggested changes, the review will come back "requested changes." This does not mean the code is bad... only that there can be some improvements to make it optimal for merging.

Testing expectations

Run tests with Vitest:

To run a specific test file (preferred)
yarn test path/to/test.spec.ts

To run every test (avoid because slow)
yarn test

To run test suites
yarn test:app for all frontend tests
yarn test:server for all backend tests
yarn test:shared for all shared code tests

Write new tests for any new features and note what they actually test. If it can be manually tested (if you can actually see if it works when running it) then please manually test as well.

AI tool use

Claude is the AI tool used and the logs are attached to the PR or added separately and attached to an issue

Definition of Done

code reviewed, tests passing, CI green, documentation updated